Diffusion-Based Generation of Novel Pokémon

with Conditional Evolution Modeling

MSML 612 - Final Presentation

Aaron Cyril John, Yugaank Kalia, Varen Maniktala

Outline

1. Problem Motivation & Recap
2. From Unconditional Diffusion Baseline to Conditional Diffusion
3. Novelty vs. Prior Work
4. Dataset & Curation Challenges
5. Preprocessing Pipeline
6. Diffusion System Architecture
7. Conditioning Design (Type / Style / Stage / Prev-Evo)
8. Training Procedure
9. Reproducibility & Code Organization
10. Results (Unconditional, Type-, Style-, Evolution-Conditioned)
11. Evaluation
12. Discussion, Limitations & Future Work

Problem Motivation

Goal: Generate novel, visually coherent Pokémon designs under structured conditional controls.

Challenge	Why It Matters
Multi-source style alignment	Official art, game sprites, and 3D renders must correspond
Attribute conditioning	Type, evolution stage, and style must guide generation
Evolution chain coherence	Generated forms must remain visually related across a chain
Domain-specific structure	Pokémon generation is not just image synthesis: it is structured conditional generation

> Standard unconditional image generators cannot capture evolution progression or type-based stylistic cues.

From Unconditional Diffusion Baseline to Conditional Diffusion

Baseline: Unconditional Diffusion

- Trains on Pokémon images only: no labels, no evolution pairs, no reference sprite
- Learns the overall sprite distribution, but cannot control **type**, **stage**, or **style**
- Generates plausible images, but cannot model **base** → **evo1** → **evo2** progression

What a conditional diffusion model gives us instead

- Uses structured records: `prev_sprite`, `next_sprite`, `types`, `evolution_stage`, `art_style`
- Conditions generation on **type**, **style**, **stage**, and optional **previous evolution image**
- Supports controlled samples: “make a fire-type base sprite” or “generate the next evolution”
- Keeps diffusion's stable denoising objective while adding user-directed control

Novelty vs. Prior Pokémon Generators

Prior work mostly generates **standalone Pokémon-like images**. Our contribution is **structured, controllable, evolution-aware generation**.

Prior Work	What It Does	Limitation We Address
Generated-Pokemon DDPM/DDIM [16]	Unconditional diffusion on 819 JPG images	No type, style, stage, or previous-evolution conditioning
Wong CS230 StyleGAN2-ADA [17]	GAN-generated sprites + separate name generation from <1K examples	No diffusion objective; no explicit evolution or attribute controls
Ours	Conditional diffusion over **6.4K curated mappings**	Generates with `types`, `art_style`, `evolution_stage`, and optional `prev_sprite`

Novel pieces in our pipeline

- Multi-source alignment across sprites, Sugimori art, and 3D renders
- Evolution-chain supervision: `prev_sprite → next_sprite`
- Unified conditioning vector for type, style, stage, timestep, and previous-evolution image

Dataset Overview

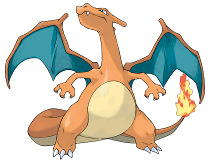
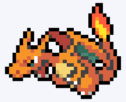
Three complementary image sources, now unified into a single diffusion-ready training set with type, stage, and style labels.

Source	Files	Role
SugimoriSprites	1,848	High-quality 2D reference art
PokeSprite	2,853	Game-style sprite (primary target domain)
ProjectPokemon 3D	2,799	Alternate visual domain for cross-format variety

Diffusion-ready mappings (with `types` + `evolution_stage` + `art_style`):

Mapping File	Entries	Role
`safe_sprite_to_sprite_types_evolution.json`	1,995	Sprite → Sprite (+ prev-evo image)
`safe_sugimori_to_sugimori_types_evolution.json`	1,674	Sugimori → Sugimori (+ prev-evo image)
`safe_project_to_project_types_evolution.json`	2,704	3D → 3D (+ prev-evo image)
Total training entries	**6,373**	Merged in `PokemonDiffusionDataset`

Data Sources: Visual Comparison

Style	Bulbasaur	Charizard
SugimoriSprites (Official Art)		
PokeSprite (Game Sprite)		
ProjectPokemon 3D		

> Each source captures a different visual style of the **same** Pokémon: cross-format alignment is critical.

Dataset Enrichment: Types + Evolution Stage

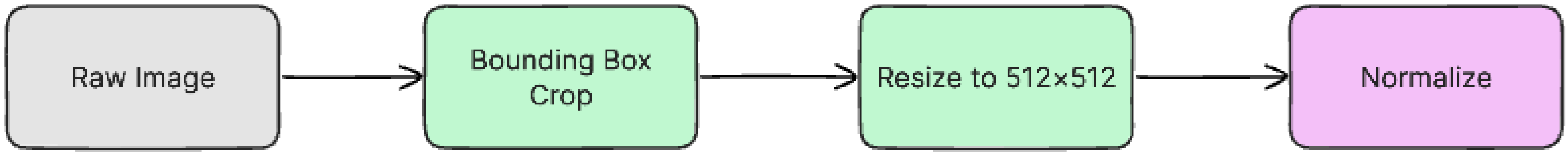
Per-entry schema (JSON):


```json
{
  "prev": null,
  "next": "bulbasaur",
  "prev_sprite": null,
  "next_sprite": "poke-data/PokeSprite/0001bulbasaur/bulbasaur.png",
  "types": ["grass", "poison"],
  "evolution_stage": "base",
  "art_style": "sprite"
},
{
  "prev": "bulbasaur",
  "next": "ivysaur",
  "prev_sprite": "poke-data/PokeSprite/0001bulbasaur/bulbasaur.png",
  "next_sprite": "poke-data/PokeSprite/0002ivysaur/ivysaur.png",
  "types": ["grass", "poison"],
  "evolution_stage": "evo 1",
  "art_style": "sprite"
}
```


Curation challenges we had to solve (not off-the-shelf):

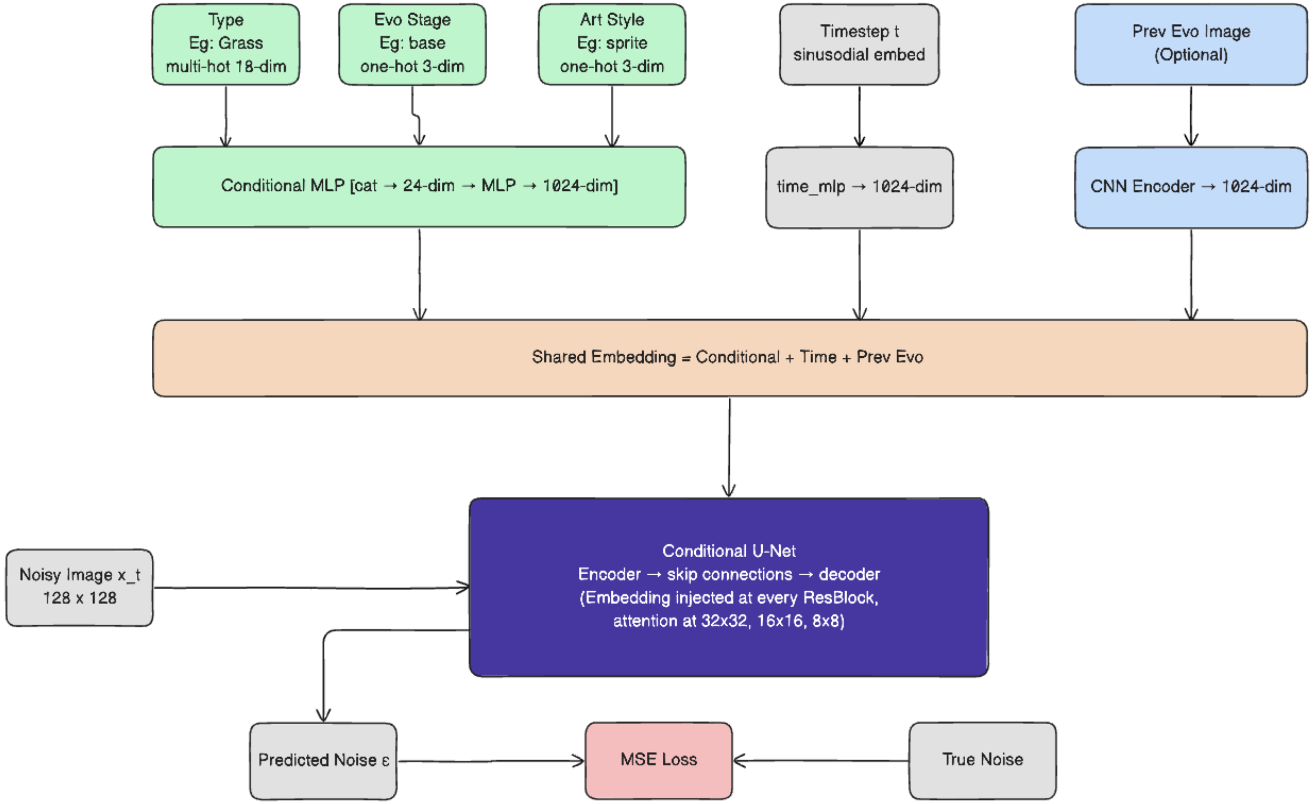
- **Cross-format name collisions**: same Pokémon, different filename conventions across sources (e.g. `0006 Charizard.png` vs `charizard.png` vs `poke_capture_0006_*.png`); resolved via normalized lookup against `poke-data/pokedex.json`
- **Regional/cosmetic variants**: `meowth-galar`, `bulbasaur shiny`, mega, gigantamax share base IDs; stripped and tagged so type lookup stays unambiguous
- **Evolution-stage resolution**: no source labels stage directly; derived via DFS over `prev → next` edges from chain roots, with PokeAPI fallback
- **Art-style labelling**: tagged per mapping file (`sprite`, `sugimori`, `3d`) so one unified dataset drives **style-conditioned** sampling

Preprocessing Pipeline



> Filename normalization → form-variant handling (Mega, Gigantamax) → cross-format mapping generation → types + evolution enrichment → RGB 128x128 resize for diffusion training

System Architecture: Conditional U-Net Diffusion



Design decisions under constraint:

- **Small dataset (~6K) → diffusion over GAN**: DDPM training is stable in low-data regimes where adversarial training collapses
- **Multi-modal conditioning** (18 types × 3 stages × 3 styles + a *reference image*) → injected via additive timestep-style modulation in every ResBlock, with a dedicated CNN encoder that compresses the prev-evo image into the same embedding space.
- **Self-attention at 32x32, 16×16, and 8×8**: captures long-range shape coherence that pure convolutions miss at this resolution
- **Unified conditioning embedding**: Each input is independently MLP-projected to a shared dimension, then `emb = t_emb + c_emb + p_emb` is broadcast to every ResBlock.

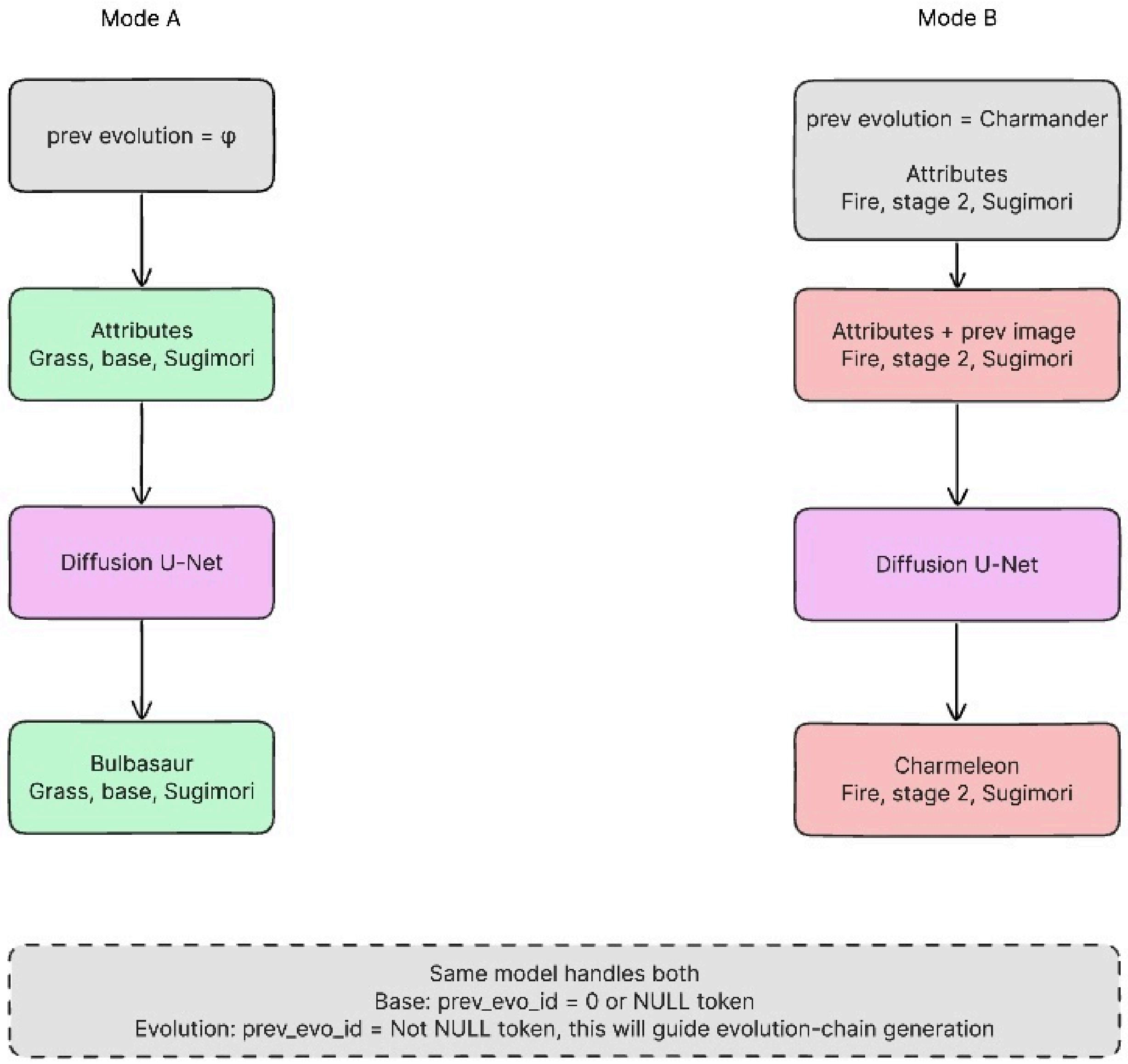
Architecture: Conditioning Design

Five conditioning signals of different types: type labels, evolution stage, art style, a reference image, and the diffusion timestep all compressed into a single vector that guides the U-Net at every layer.

Conditioning Input	Encoding	Why this encoding
Pokémon Type (e.g., Fire, Water)	Multi-hot over 18 types (supports dual-type)	One-hot would lose dual-typing entirely
Evolution Stage (base / evo1 / evo2)	One-hot over 3 stages	Discrete structural complexity control
Visual Style (3d / sugimori / sprite)	One-hot over 3 styles	Enables cross-domain sampling at inference
Previous Evolution Image	CNN encoder → dense vector (zeroed if absent)	Base-stage Pokémon have no prior; zero-masking via `has_prev` keeps batch shape valid
Timestep `t`	Sinusoidal embedding → MLP	Standard DDPM timestep encoding

> Each signal is independently MLP-projected to a shared dimension, then **summed** (`emb = t_emb + c_emb + p_emb`) into a single embedding vector, which is injected additively into every ResBlock at every resolution via a learned linear projection - not just at the bottleneck.

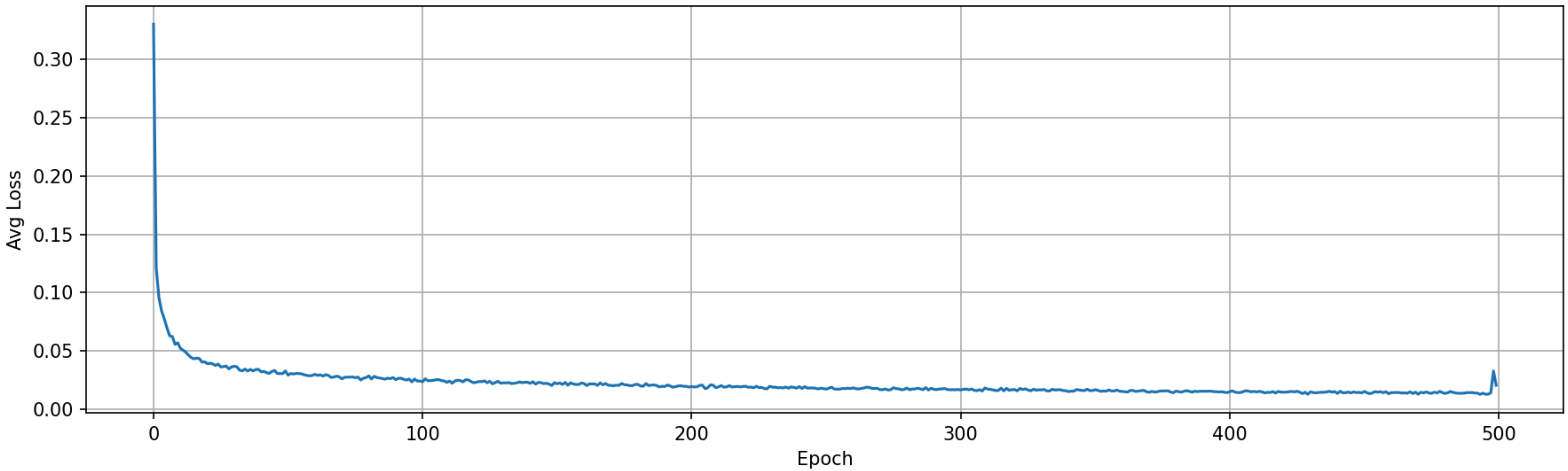
Training Procedure



Training Details

Component	Setting
Model	Conditional U-Net, self-attention at 32, 16, & 8
Parameters	~21 M
Dataset	6,373 entries merged across `3d` / `sugimori` / `sprite` mappings
Hardware	G4 High-RAM GPU: NVIDIA RTX PRO 6000 Blackwell (Google Colab)
Training time	~5 hours for 500 epochs

Loss Plot



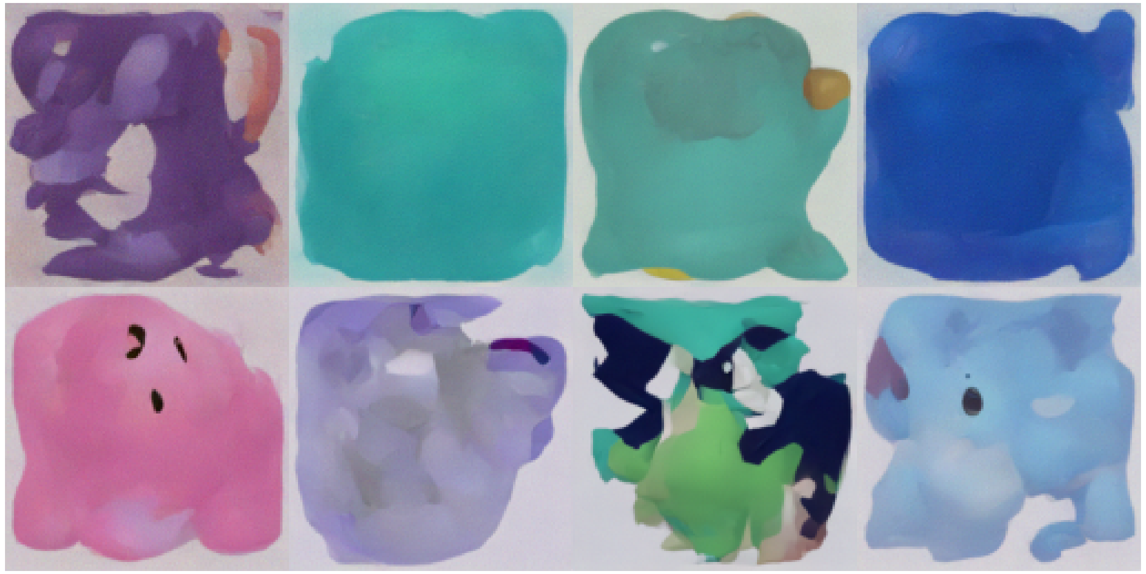
Reproducibility & Code Organization

The full pipeline reproduces end-to-end from a clean checkout with a single command.

Concern	How we handle it
Deterministic runs	Deterministic `DataLoader` workers with dataset sampler to avoid dataset imbalance
Data loading	Single `PokemonDiffusionDataset` class; unifies all 3 mapping files
Entry point	`python main.py` reproduces training via `--train` or inference via `--inference`
Environment	`requirements.txt`; tested on NVIDIA G4 (Colab) and HPC
Checkpoints	EMA + raw weights saved every N epochs; hardcoded resumable option
Sampling	`python main.py --inference --ckpt ... --type fire --stage base --style sprite`

Diffusion Results: Unconditional Samples

3D



Sprites

Diffusion Results: Type-Conditioned Samples

3D

Sprites

Diffusion Results: Style-Conditioned Samples

3 Styles (3d, sugimori, sprite)

Evaluation Plan

Metric	Description
FID (Fréchet Inception Distance)	Distribution distance between real and generated images (lower = more realistic)
KID (Kernel Inception Distance)	Same as FID but unbiased for small N (2000 samples) (more reliable than FID at smaller dataset size)
ID (Intra-class Distance)	Average pairwise distance between generated images
Qualitative Review	Side-by-side visual inspection

Quantitative Results

Metric	Unconditional Diffusion Baseline	Conditional Diffusion (Single Style)	Conditional Diffusion (Multiple Styles)
FID	123	**77**	202
KID	0.132 ± 0.002	**0.061 ± 0.0017**	0.231 ± 0.033
ID	15.44	**16.23**	17.39

Discussion

What worked well

- Unified multi-source dataset (~6.4K entries) with clean `types` / `stage` / `style` labels

What was hard

- Cross-format name normalization (shiny, mega, gigantamax, regional variants)
- Preprocessing took a large majority of time since naming, orientations, and structures were inconsistent across art styles
- Small dataset relative to typical diffusion training, so careful augmentation & EMA were important

Limitations & Future Work

Current limitations

- Trained at **128×128 RGB**; fine sprite detail still bounded by resolution
- Only 3 styles and 3 evolution stages; real Pokémon have far more visual variants
- No text conditioning (e.g. natural-language prompts for abilities or lore)
- Our code currently produces evo chains (pokemon images based on previous pokemon evos), but they lack visual coherence (structure)

Future directions

- **Scale up resolution** with a latent diffusion / super-resolution stage
- **Text conditioning** using a pretrained text encoder (e.g. CLIP) for descriptive generation
- **Full evolution-chain consistency loss** that ties together all generated stages jointly
- **RGBA masking** Using 4 channel inputs (RGBA) for creating correct transparency masking to produce clean RGB images

Conclusion

- Moved from an **unconditional diffusion baseline** to a **conditional diffusion generator**
- Built a 6.4K-entry diffusion-ready dataset with type, stage, and style metadata across sprite, Sugimori, and 3D sources
- Implemented a **Conditional U-Net** with a dedicated prev-evolution image encoder
- Demonstrated type-, style-, and stage- conditioned generation of novel Pokémon

> Conditional diffusion is a substantially better fit than the unconditional baseline for the structured, low-data, multi-domain setting of Pokémon generation.

References

1. Ho, J., Jain, A., Abbeel, P. *Denoising Diffusion Probabilistic Models.* NeurIPS, 2020. arxiv.org/abs/2006.11239
2. Nichol, A., Dhariwal, P. *Improved Denoising Diffusion Probabilistic Models.* ICML, 2021. arxiv.org/abs/2102.09672
3. Dhariwal, P., Nichol, A. *Diffusion Models Beat GANs on Image Synthesis.* NeurIPS, 2021. arxiv.org/abs/2105.05233
4. Rombach, R., Blattmann, A., Lorenz, D., Esser, P., Ommer, B. *High-Resolution Image Synthesis with Latent Diffusion Models.* CVPR, 2022. arxiv.org/abs/2112.10752
5. Karras, T., Aittala, M., Aila, T., Laine, S. *Elucidating the Design Space of Diffusion-Based Generative Models.* NeurIPS, 2022. arxiv.org/abs/2206.00364
6. Ho, J., Salimans, T. *Classifier-Free Diffusion Guidance.* NeurIPS Workshop, 2021. arxiv.org/abs/2207.12598
7. Saharia, C., Chan, W., Saxena, S., et al. *Palette: Image-to-Image Diffusion Models.* SIGGRAPH, 2022. arxiv.org/abs/2111.05826
8. Ronneberger, O., Fischer, P., Brox, T. *U-Net: Convolutional Networks for Biomedical Image Segmentation.* MICCAI, 2015. arxiv.org/abs/1505.04597
9. Perez, E., Strub, F., de Vries, H., Dumoulin, V., Courville, A. *FiLM: Visual Reasoning with a General Conditioning Layer.* AAAI, 2018. arxiv.org/abs/1709.07871
10. Isola, P., Zhu, J.-Y., Zhou, T., Efros, A. A. *Image-to-Image Translation with Conditional Adversarial Networks (pix2pix).* CVPR, 2017. arxiv.org/abs/1611.07004
11. Hugging Face. *Diffusers: State-of-the-art diffusion models in PyTorch.* github.com/huggingface/diffusers
12. PokéAPI. pokeapi.co
13. msikma. *pokesprite.* github.com/msikma/pokesprite
14. Project Pokémon. Sprite and asset resources. projectpokemon.org
15. Pokémon Database. National Pokédex and image resources. pokemondb.net/pokedex/national
16. Followb1ind1y. *Generated-Pokemon: New Pokemon Generation using Diffusion Models.* github.com/Followb1ind1y/Generated-Pokemon
17. Wong, R. *Pokémon Generator.* Stanford CS230, 2021. cs230.stanford.edu/projects_fall_2021/reports/103146257.pdf